

Operators



Logical

!a	not a
a && b	a and b
a b	a or b

Shift

a << b	a shift left by b
a >> b	a shift right b

Reduction

&a	reduction and
a	reduction or

Bit-Wise

~a	not a
a & b	a and b
a b	a or b
a ^ b	a xor b
a ~^ b	a xnor b
a ^~ b	

Conditional

condition ? True (exp1) : False (exp2)

Operators



Arithmetic

$a + b$	a plus b
$a - b$	a minus b
$a * b$	a multiply by b
a / b	a divide by b
$a \% b$	a modulo b

Relational

$a < b$	a less than b
$a > b$	a greater than b
$a \leq b$	a less than or equal to b
$a \geq b$	a greater than or equal to b
$a \neq b$	a not equal to b, including x and z
$a == b$	a equal to b, may be unknown
$a != b$	a not equal to b, may be unknown

Continuous Assignment Examples



```
wire [3:0] A, X, Y, R, Z;  
wire [7:0] P;  
wire r, a, cout, cin;
```

```
assign R = X | (Y & ~Z);
```

```
assign r = &X;
```

example
reduction
operator

use of bit-wise Boolean operators

```
assign R = (a == 1'b0) ? X : Y;
```

conditional operator

```
assign P = 8'hff;
```

example constants

```
assign P = X * Y;
```

arithmetic operators (use with care!)

```
assign P[7:0] = {4{X[3]}, X[3:0]};
```

(ex: sign-extension)

```
assign {cout, R} = X + Y + cin;
```

bit field concatenation

```
assign Y = A << 2;
```

bit shift operator

```
assign Y = {A[1], A[0], 1'b0, 1'b0};
```

equivalent bit shift

Verilog – Data flow Description



```
module multiplexer (A, B, Sel, Out)
```

```
input A, B, Sel;
```

```
output Out;
```

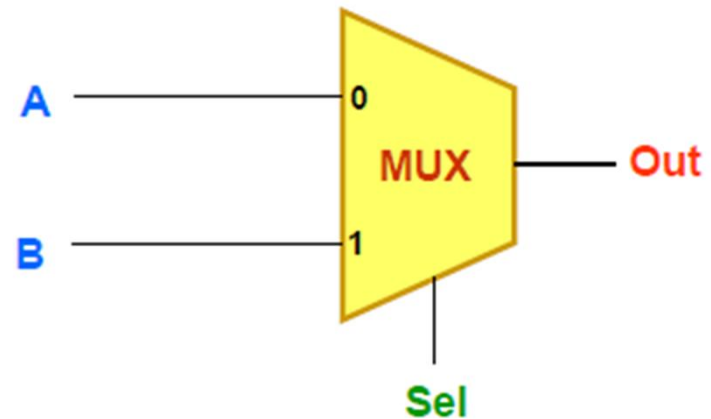
```
// using conditional operators
```

```
assign Out = Sel ? B : A;
```

```
// using bit-wise operators
```

```
assign Out = A & ~Sel | B & Sel;
```

```
endmodule
```



$$Out = \overline{Sel} \cdot A + Sel \cdot B$$

Modeling with vector data (multiple bit widths)



- A **vector** is specified within square brackets and two numbers separated with a colon.
- e.g. **output[0:3]D;** - This declares an output vector D with 4 bits, 0 through 3.
- e.g. **wire[7:0]SUM;** – This declares a wire vector SUM with 8 bits numbered 7 through 0.
- The **first number** listed is the most significant bit (**MSB**) of the vector.

Dataflow description of 4-bit adder



```
module binary_adder (A,B,Cin,SUM,Cout);  
    input [3:0] A,B;  
    input Cin;  
    output [3:0] SUM;  
    output Cout;  
    assign {Cout,SUM} = A + B + Cin;  
endmodule
```

Dataflow description of a 4-bit comparator



```
module magcomp (A,B,ALTB,AGTB,AEQB);  
    input [3:0] A,B;  
    output ALTB,AGTB,AEQB;  
    assign ALTB = (A < B),  
           AGTB = (A > B),  
           AEQB = (A == B);  
endmodule
```

4-to-1 Multiplexer – Data Flow



//4-to-1 Mux: Boolean Expression

```
module mux_4_to_1(S,D,Y);  
    input [1:0]S;  
    input [3:0]D;  
    output Y;  
    assign Y = (~S[1]&~S[0]&D[0])|(~S[1]&S[0]&D[1])  
               |(S[1]&~S[0]&D[2])|(S[1]&S[0]&D[3]);  
endmodule
```


4-to-1 Multiplexer – Data Flow



//4-to-1 Mux: Truth table based conditional

```
module mux_4_to_1(S,D,Y);  
    input [1:0]S;  
    input [3:0]D;  
    output Y;  
    assign Y=S[1]?(S[0]?D[3]:D[2]):(S[0]?D[1]:D[0]);  
endmodule
```

//4-to-1 Mux: Conditional Dataflow description

```
module mux_4_to_1(S,D,Y);  
    input [1:0]S;  
    input [3:0]D;  
    output Y;  
    assign Y = (S==2'b00)?D[0] : (S==2'b01)?D[1] :  
    (S==2'b10)?D[2] : (S==2'b11)?D[3]:1'bx;;  
endmodule
```